

Lab 01 Tasks

Introductory Computer Vision Lab (Extended: 2 Hours)

Part 1: Environment Setup & Validation

Before beginning the tasks, ensure your notebook environment is correctly configured to handle image processing and plotting.

- **1.1 Installation:** Use Jupyter magic commands to install the required libraries by running `%pip install opencv-python` and `%pip install matplotlib` in your first cell.
- **1.2 Import & Validate:** Import `cv2` and `matplotlib.pyplot` as `plt`. Write a quick validation script to ensure they are working properly before proceeding.

Part 2: Python Data Structures & Logic

- **Task 1: Object-Oriented Grocery Manager**

Create a `GroceryManager` class instead of a simple script.

- Implement methods to `add_item(item, quantity, price)`, `remove_item(item)`, and `view_list()`.
- Add a method `calculate_total()` that computes the total cost of all items in the list.
- Implement basic error handling (e.g., preventing the removal of an item that doesn't exist).

- **Task 2: Advanced Student Record System**

Build a nested dictionary-based record system.

- Each student `ID` should map to a dictionary containing their `'Name'`, `'Major'`, and a list of `'Grades'`.
- Write a function that accepts this nested dictionary and returns the name of the student with the highest average grade.
- Write a search function that takes a major as input and prints all students enrolled in that major.

Part 3: Core Computer Vision & Matplotlib

All image visualization tasks must utilize Matplotlib subplots for side-by-side comparisons unless stated otherwise.

- **Task 3: Safe Image Loading & RGB Visualization (10 Mins)**

Standard `OpenCV` reads images in `BGR` format, which displays with inverted colors in Matplotlib.

- Load an image from your directory (e.g., `image_path = 'Sukuna.jpeg'`) using `cv2.imread()`.
 - Implement a safety check using `if image is None:` to print an error message (e.g., `"Error: Image not found."`) if the path is incorrect.
 - If the image loads successfully, convert it from BGR to RGB using `cv2.cvtColor(image, cv2.COLOR_BGR2RGB)`.
 - Set up your plot using `plt.figure(figsize=(8, 6))` and hide the axes using `plt.axis('off')`.
 - Display the image using `plt.imshow()` and `plt.show()`, and add a highly formatted title (e.g., `plt.title('King of Curses - Ryomen Sukuna', fontsize=16, color='darkred', pad=15)`).
- **Task 4: Interactive Geometry & Blank Canvas**
 - Create a `800x800` blank black image using `NumPy`.
 - Use `OpenCV` to draw a geometric pattern (e.g., a target board with `5` concentric circles of alternating colors).
 - Add a bounding box (rectangle) around the outermost circle and calculate/print the exact center coordinates of the canvas.
- **Task 5: Advanced Blurring & NumPy ROI Extraction**
 - Load a high-resolution image (safely check if it loaded, convert to RGB).
 - Apply a heavy `Gaussian Blur` (e.g., `25x25` kernel).
 - Using `NumPy` array slicing, extract a `300x300` pixel Region of Interest (ROI) from the *exact center* of both the original and blurred images.
 - Plot the two ROIs side-by-side using Matplotlib subplots, ensuring axes are turned off with `plt.axis('off')` and custom

titles are added to each subplot using `fontsize` and color parameters.

- **Task 6: Alpha Blending & Typography**

- Create a stylized image caption. Load an image and create a solid color overlay (e.g., a blue rectangle) that covers the bottom `20%` of the image.
- Blend the original image and the rectangle overlay using `cv2.addWeighted()` so the overlay is semi-transparent.
- Use `cv2.putText()` to write a title over the transparent box.

- **Task 7: Matrix-Based Rotation & Adaptive Thresholding**

- Load a grayscale document or high-contrast image.
- Instead of basic binary thresholding, apply both Global Binary Thresholding and **Adaptive Thresholding** (`cv2.adaptiveThreshold`). Compare them side-by-side.
- Use `cv2.getRotationMatrix2D()` to create a rotation matrix, and rotate the adaptive threshold output by exactly `45` degrees, scaling it by `0.8` so the corners don't get cut off.

- **Task 8: Masking with Bitwise Logi**

- Load two different images and resize them to be exactly the same dimensions (e.g., `500x500`).
- Create a binary mask (a black image with a thick white polygon or circle in the center).
- Use `cv2.bitwise_and` to cut out the polygon shape from the first image.
- Use `cv2.bitwise_not` to invert the mask, and use it to cut out the *background* from the second image. Combine them using `cv2.bitwise_or`.

- **Task 9: Statistical Image Profiling with Pandas**

- Load an image and flatten its 2D pixel arrays for the **Red**, **Green**, and **Blue** channels into `1D` arrays.
- Load these three arrays into a **Pandas** DataFrame with columns `['Red', 'Green', 'Blue']`.

- Use Pandas built-in functions (`.describe()`) to analyze the `mean`, `min`, `max`, and standard deviation of each color channel. Print the `DataFrame` summary to the console.